

CarbonLedger

Configuration Management Plan

Joint versioning of calculation code and reference data

CL-CMP-001 · VERSION 1.0 · DRAFT FOR REVIEW

1

repository

Pair

code + data

SHA-256

manifest hash

Prepared by: Rohan Vashisht

Cohort: Jensen Huang

Roll No.: 150096724132

Programme: B.Tech CSE 2024–28 · Semester V

Project: Emissions Accounting for a Multi-Plant Manufacturer

Project website: <https://carbon-ledger-rohan.vercel.app/>

Summary & Contents

Purpose of this document

Defines how code and reference data are versioned, baselined, changed, archived and audited together, so that a report is reproducible as a pair and not just as code.

Contents

1	Purpose and scope	4
2	Configuration items	4
3	Joint versioning of code and reference data	5
4	Repository, branching and baselines	6
5	Change control	7
6	Build and environment control	8
7	Status accounting and records	8
8	Configuration audits	8
9	Archive, backup and retention	9
10	Roles	9
11	Risks to configuration integrity	9

Document control

Revision history

Version	Date	Description	Author (role)
1.0	03/10/2026	Initial release for review	Sole Engineer

Approvals

Role	Signature	Date
Sponsor (Manufacturer, Head of Sustainability)		
Reporting Lead (Manufacturer)		
Sole Engineer (all project roles)		
Reviewer (Faculty)		

Related documents

- CL-SRS-001 Software Requirements Specification (v1.0)
- CL-DTT-001 Decision Table and Test Set (v1.0)
- CL-DSG-001 Design Pack (v1.0)
- CL-CMP-001 Configuration Management Plan (v1.0) · this document
- CL-DQP-001 Data Quality Plan (v1.0)
- CL-PPL-001 Project Plan (v1.0)

Notice. This document is controlled; printed copies are uncontrolled. Figures stated as project data (data quality tiers of ±2%, ±8% and ±25%, nine plants, three scopes) come from the case study. Numbers in worked examples and test fixtures are synthetic, are labelled as such, and are not emission factors published by any authority.

1. Purpose and scope

This plan defines how CarbonLedger's **code and reference data are versioned together**, so that an issued report is reproducible as a **pair** (calculation logic plus reference data), not just as code. It covers identification, version control, baselines, change control, build and environment control, archive, and audit.

The central rule: *a report is only meaningful together with the exact calculation logic and the exact reference data that produced it. Both are recorded as one Pair, tagged once, stored once, and verified together.*

Why code-only versioning is not enough: if the code is tagged but the factor table is a live database table, checking out the 2024 tag in 2026 still reads the 2026 factors, and the 2024 figure changes. If the data is versioned but the code is not, a rounding fix silently changes old results. The Pair removes both gaps.

2 Configuration items

ID	Configuration item	Location	Identification	Change control class
CI-01	Application source code	<code>src/</code>	Git commit and Pair tag	A if it can alter a number, else B
CI-02	Calculation modules (<code>calc/v1</code> , <code>calc/v2</code> ...)	<code>src/carbonledger/calc/</code>	Module name; frozen once used by an issued report	A
CI-03	Reference releases (factors, conversions, tier percentages, rounding policy)	<code>refdata/RR-<year>.<n>/</code>	Release id and manifest SHA-256	A
CI-04	Database migrations	<code>migrations/</code>	Sequence number; checksum	A for constraints and triggers
CI-05	Pair lock files	<code>pairs/CL-<year>.<n>.lock</code>	Pair id; lock hash	A
CI-06	Golden test artefacts	<code>tests/golden/R-*/</code>	Report id	A (add only; never edit)
CI-07	Dependency lock file and base image digest	<code>requirements.lock</code> , <code>Dockerfile</code>	Content hash; image digest	A
CI-08	Issued reports (canonical JSON, PDF, CSV) and input snapshots	Archive store	Report id and version, SHA-256	Immutable
CI-09	Evidence files	Evidence store	SHA-256	Immutable
CI-10	Project documents (SRS, DTT, Design Pack, CMP, DQP, Project Plan)	<code>docs/</code>	Document id and version	B

Class **A** = may change a reported number or the ability to reproduce one; **B** = cannot (user interface text, documents).

3. Joint versioning of code and reference data

3.1 One repository

Code and reference data live in **one Git repository**. A reference release is a directory of plain files (`factors.csv`, `conversions.csv`, `policy.json`) plus a `manifest.json` listing the SHA-256 of every row and file. Plain files are human-diffable, reviewable and hashable, and they are loaded into PostgreSQL by a migration that re-verifies the hashes.

3.2 The Pair

A **Pair** is the unit of reproducibility. Its lock file records everything needed to re-run it:

```
pair_id:          CL-2024.1
calc_module:      carbonledger.calc.v1
git_commit:       <40 hex characters>
refdata_release:  RR-2024.1
refdata_sha256:   <64 hex characters>
requirements_lock_sha256: <64 hex characters>
python:           3.12.x
container_digest: sha256:<64 hex characters>
created:          2025-01-10T09:00:00Z
```

An annotated Git tag `CL-2024.1` points at the commit that contains the code, the reference release and this lock file together. Checking out the tag gives both halves at once. The lock is copied into the `release_pair` table, and every report row stores its `pair_id`.

3.3 Naming and numbering

Item	Scheme	When it changes
Reference release	RR-<year>.<n>	Every new publication or correction by the authority. Never edited after sealing.
Calc module	calc.v<N>	Only when a change could alter any reported number (formula, rounding, apportioning, gap-fill rule, uncertainty method).
Pair	CL-<year>.<n>	Whenever either half changes: a new reference release, or a new calc module.
Application version	MAJOR.MINOR.PATCH	Releases of the web application. A PATCH or MINOR change that touches no calc module and no reference data keeps the current Pair.

Illustration (synthetic):

Pair	Calc module	Reference release	Used for
CL-2024.1	calc.v1	RR-2024.1	FY2024 reports
CL-2025.1	calc.v1	RR-2025.1	FY2025 reports (new factors, same logic)
CL-2026.1	calc.v2	RR-2026.1	Reports after a method change or a factor correction

Old Pairs are never deleted or edited. Several Pairs are always runnable side by side.

3. Joint versioning of code and reference data

3.4 Rules that keep the Pair honest

1. A calc module used by an issued report is **frozen**: CI fails any commit that changes its files (checked against the hash in the lock file).
2. A sealed reference release is **frozen**: CI fails any commit that changes its files; the database rejects changes to sealed rows.
3. A bug found in a frozen module is **not fixed in place**. The fix goes into a new module and a new Pair. Old results stay reproducible, including the defect, and are changed only by an explicit restatement (SRS FR-LATE-06). The known defect is listed in the release notes.
4. Every report records its Pair; **verify** refuses to run a report with any other Pair.
5. Reference data and calc changes may not be applied by editing the database. The only path is repository change, review, CI and tag.

4

Repository, branching and baselines

4.1 Branching

Trunk-based: a protected **main**, short-lived branches named **feat/<id>**, **fix/<id>**, **ref/<release>**, **docs/<id>**. Commits use the form **<type>(<area>): <summary> [CR-*nnn*]**. Merge is only by pull request with a green pipeline. With a single engineer, the pipeline and the checklist in section 5.3 stand in for a second reviewer; this is recorded as a risk (CL-PPL-001, R-04) and mitigated by automation.

4.2 Baselines

Baseline	Content	Established	Tag
BL-1 Requirements	SRS v1.0, DTT v1.0	End of W-18 (milestone M1)	BL-REQ-1.0
BL-2 Design	Design Pack v1.0, CMP v1.0	End of W-16	BL-DES-1.0
BL-3 First Pair	Code with calc.v1 and the first reference release; golden tests passing	End of W-14 (M3)	CL-<year>.1 (first)
BL-4 Release candidate	All features; test suite green; UAT version	End of W-3 (M5)	rc-1.0
BL-5 Production Pair	Frozen Pair used for the first issued report	Before production close (W-2)	CL-<year>.<n>
BL-6 Issued report	Report, hash, inputs snapshot, golden directory added	At issue (end of W-1, M6)	R-<id>-v1

5. Change control

5.1 Change request flow

Raise change request (CR) with class A or B, reason and impact; assess against the SRS and golden tests; approve (Sponsor for scope and calc changes, Sole Engineer for class B); implement on a branch; CI; merge; tag; record in the change log. Every CR number appears in the commit messages.

5.2 Evidence required by class

Class	Required before merge
A, calc change	New module <code>calc.v<N></code> ; all golden tests pass unchanged for all older Pairs; new golden for the new Pair; release notes stating what changes and for whom.
A, reference data	Diff of the new release against the previous one, reviewed row by row against the authority's publication; manifest hash; all existing golden tests pass unchanged (a new release must not change old reports).
A, migration	Migration test on a copy of the production database; immutability triggers verified.
B	Golden test output unchanged (an empty diff proves no number moved).

5.3 Pre-merge checklist (used instead of a second reviewer)

1. CI green: unit, decision-table (48 rows), integration and all golden tests.
2. No file of a frozen calc module or sealed release changed.
3. Pair lock updated if and only if a calc module or reference release changed.
4. Class assigned correctly (a class B label on a change that moves a golden file is rejected automatically).
5. Release notes written.

5.4 Adding a reference release (procedure)

1. Steward obtains the authority's publication and records its edition.
2. Steward prepares `factors.csv` and `conversions.csv` in `refdata/RR-<year>.<n>/`; tool computes `manifest.json`.
3. Tool prints the diff against the previous release; Steward and Reporting Lead check changed rows against the publication.
4. Pull request on branch `ref/<release>`; CI verifies the manifest, checks intervals do not overlap and every type/unit has coverage for the activity dates in use, and runs all golden tests.
5. Merge, create the new Pair lock and tag; migration loads the release and **seals** it.
6. Existing reports are unaffected. If the new release corrects an old year, a what-if comparison is prepared and the Reporting Lead decides whether to restate.

6. Build and environment control

Aspect	Control
Dependencies	<code>requirements.lock</code> with hashes (installs fail on any mismatch); lock hash stored in the Pair.
Runtime	Container image built from a base image pinned by digest; digest stored in the Pair.
Time and locale	Container runs in UTC with a fixed locale; the clock is injected into code, never read directly in calculation.
Build	Same pipeline for every tag; builds are repeatable from the tag.
Determinism check	CI runs the golden suite twice, with different <code>PYTHONHASHSEED</code> , time zone and input order; outputs must be byte-equal (TC-RP-03).
Defect in a frozen module	Handled as in rule 3 of section 3.4: new module, new Pair, restatement if needed.

7 Status accounting and records

- **Change log:** every CR with class, decision, commits and Pair affected.
- **Pair register:** table `release_pair` and the `pairs/` directory, listing every Pair, its halves, dates and the reports issued under it.
- **Report register:** report id, version, period, Pair, `closed_at`, hashes, status (DRAFT, ISSUED, SUPERSEDED).
- **Release notes:** for every tag, including known defects.

The registers are produced from Git and the database by script, not by hand.

8 Configuration audits

Audit	When	Method	Pass condition
Functional audit	Every commit and every tag	Golden suite (TC-RP-04, TC-RP-05)	All old reports recompute byte for byte.
Physical audit	System start, every release load, every <code>verify</code>	Recompute SHA-256 of release files, lock file, inputs and reports	All equal the recorded hashes.
Baseline audit	Each baseline in 4.2	Check content against the baseline list	Complete and tagged.
Reproduction drill	Once a year	Restore the archive on a fresh machine and <code>verify</code> every issued report	All PASS.

9. Archive, backup and retention

Item	Retained	Where	Notes
Git repository with all tags	At least two years after the last report issued under the Pair, or the contract period if longer	Remote repository plus an offline bundle at each Pair tag	The bundle protects against loss of the hosting service.
Container image for each Pair	Same	Registry plus exported tar archive	Allows running the exact environment again.
Issued reports, inputs snapshots, golden directories	Same	Archive store with SHA-256 manifest	Immutable.
Database	Daily backup; restore tested every quarter	Separate storage	A backup that has never been restored is not counted.
Evidence files	Same as reports	Evidence store	Hash stored on each record.

10 Roles

Role	Responsibility
Sole Engineer, as Configuration Manager	Runs this plan, tags Pairs, keeps registers.
Sole Engineer, as Developer and Tester	Implements changes, maintains golden tests.
Reference Data Steward (Manufacturer)	Prepares and checks reference releases.
Reporting Lead (Manufacturer)	Approves close and restatement; owns the report register.
Sponsor (Manufacturer)	Approves class A scope changes.
Auditor	Read-only; may run verify and request the Pair register.

11 Risks to configuration integrity

Risk	Control
Someone edits a factor directly in the database	Sealed releases, database role without rights, immutability triggers, physical audit.
Library update changes arithmetic or serialisation	Hash-locked dependencies, pinned image, determinism check in CI.
Only one engineer can approve changes	Automated gates and checklist; offline bundle and documented procedures for handover.
Pair tag lost or moved	Annotated tags, offline bundle, lock hash stored in the database and in each report.
Calc defect found after issue	Frozen module, new module, release notes, restatement rule.